Lab 2: A Neural Network from Scratch

Alejandro Jimenez

Network design:
I used two hidden layers with ReLU and an output layer with softmax. The input has 64 features and the output has 10 classes

For hidden width $W = 16$

Input 64 → Linear 16 → ReLU → Linear 16 → ReLU → Linear 10 → Softmax

For hidden width $W = 64$:

Input 64 → Linear 64 → ReLU → Linear 64 → ReLU → Linear 10 → softmax

The parameter shapes are:

Parameter	Hidden width 16	Hidden width 64
W_1	64×16	64×64
b_1	$(16,)$	$(64,)$
W_2	16×16	64×64
b_2	$(16,)$	$(64,)$
W_3	16×10	64×10
b_3	$(10,)$	$(10,)$

I initialized the weights from $N(0, 0.01)$, interpreting 0.01 as the variance. The standard deviation is therefore

$$\sigma = \sqrt{0.01} = 0.1$$

All biases start at zero.

Forward equations:

Each row of X represents one sample. For a batch of B sample: $X \in \mathbb{R}^{B \times 64}$

First hidden layer: $Z_1 = XW_1 + b_1$ $\quad A_1 = ReLU(Z_1)$

Second hidden layer: $Z_2 = A_1 W_2 + b_2$ $\quad A_2 = ReLU(Z_2)$

Output layer: $Z_3 = A_2 W_3 + b_3$

For sample i and class K, softmax gives: $P_{iK} = \dfrac{e^{Z_{3,iK}}}{\sum_{j=1}^{10} e^{Z_{3,ij}}}$

Using one-hot targets Y, the mean cross-entropy loss is: $L = -\frac{1}{B} \sum_{i=1}^{B} \sum_{K=1}^{10} Y_{iK} \log P_{iK}$

Softmax and cross-entropy gradient:

For one sample, I write the loss as: $\ell = -\sum_{j=1}^{10} y_j \log p_j$

Its derivative with respect to p_j is: $\dfrac{\partial \ell}{\partial p_j} = -\dfrac{y_j}{p_j}$

The softmax derivative is: $\dfrac{\partial p_j}{\partial z_K} = p_j(\delta_{jK} - p_K)$ Here, $\delta_{jK} = 1$ when $j = K$, and 0 otherwise.

Applying the chain rule:

$$\frac{\partial \ell}{\partial z_K} = \sum_{j=1}^{10} \frac{\partial \ell}{\partial p_j} \frac{\partial p_j}{\partial z_K} = \sum_{j=1}^{10} \left(-\frac{y_j}{p_j}\right) p_j(\delta_{jK} - p_K) = -y_K + p_K \sum_{j=1}^{10} y_j$$

Alejandro Jimenez

Since the target is one-hot, $\sum_j y_j = 1$. Therefore: $\frac{\partial \ell}{\partial z_k} = p_k - y_k$

For the mean loss over a batch: $D_3 = \frac{\partial L}{\partial Z_3} = \frac{P - Y}{B}$

Output-layer parameters:

The output layer is: $Z_3 = A_2 W_3 + b_3$

For one weight $W_{3,jk}$, the chain rule gives: $\frac{\partial L}{\partial W_{3,jk}} = \sum_{i=1}^{B} \frac{\partial L}{\partial Z_{3,ik}} \frac{\partial Z_{3,ik}}{\partial W_{3,jk}}$

Since $\partial Z_{3,ik} / \partial W_{3,jk} = A_{2,ij}$ $\quad \frac{\partial L}{\partial W_{3,jk}} = \sum_{i=1}^{B} A_{2,ij} D_{3,ik}$

In matrix form: $\frac{\partial L}{\partial W_3} = A_2^T D_3$ $\quad$ Each bias is shared across the batch: $\frac{\partial L}{\partial b_{3,k}} = \sum_{i=1}^{B} D_{3,ik}$

The gradient passed to the second hidden layer is: $\frac{\partial L}{\partial A_2} = D_3 W_3^T$

Second hidden layer:

For ReLU, I use: $\mathrm{ReLU}'(z) = \begin{cases} 1, & z > 0 \\ 0, & z \le 0 \end{cases}$ $\quad$ At zero, I use a derivative of zero, matching the implementation.

Since $A_2 = \mathrm{ReLU}(Z_2)$, the chain rule gives: $D_2 = \frac{\partial L}{\partial Z_2} = \frac{\partial L}{\partial A_2} \odot \mathrm{ReLU}'(Z_2)$

Therefore: $D_2 = (D_3 W_3^T) \odot 1(Z_2 > 0)$ $\quad$ Here, $\odot$ means element-wise multiplication.

Using $Z_2 = A_1 W_2 + b_2$: $\frac{\partial L}{\partial W_2} = A_1^T D_2$ $\quad \frac{\partial L}{\partial b_{2,j}} = \sum_{i=1}^{B} D_{2,ij}$

The gradient passed to the first hidden layer is: $\frac{\partial L}{\partial A_1} = D_2 W_2^T$

First hidden layer:

Since $A_1 = \mathrm{ReLU}(Z_1)$: $D_1 = \frac{\partial L}{\partial Z_1} = \frac{\partial L}{\partial A_1} \odot \mathrm{ReLU}'(Z_1)$ Therefore: $D_1 = (D_2 W_2^T) \odot 1(Z_1 > 0)$

Using $Z_1 = X W_1 + b_1$: $\frac{\partial L}{\partial W_1} = X^T D_1$ $\quad \frac{\partial L}{\partial b_{1,j}} = \sum_{i=1}^{B} D_{1,ij}$

These equations give the gradients for all three weight matrices and all three bias vectors. They apply to both hidden widths. The batch average is already included in D_3, so I do not divide by B again in the earlier layers.